CS 131 PA3 Technical Report

Team 6, CineMatch-Netflix-Movies: Aneesh, Krishna, Hadi, Pouya, Prajwal. Branches: `pa3-pipeline` and `pa3-benchmark-schedule`. Everything was run on the SJSU IBM server.

1. Pipeline contract

The question this pipeline answers is which crew roles show up in the top 10 billing spots of IMDb titles, and how high they are billed on average. CineMatch recommends movies based on who worked on them, so this tells us which roles are common and important enough to use as features in the final project, and which ones are too rare to bother with.

The raw input is `imdb_data/title.principals.tsv` (4.3 GB). It is tab-separated with one header row and 6 fields: `tconst, ordering, nconst, category, job, characters`. We use `tconst, ordering, nconst, category`. For Task 9 we also use `tconst` and `startYear` from `title.basics.tsv`.

The analysis-ready table is `work/analysis_ready.tsv`, tab-separated with a header. Its columns are `title_id` (tt plus digits), `billing_order` (an integer of at least 1), `person_id` (nm plus digits), and `role_category` (text, never `\N`). The shell, DuckDB-text, and DuckDB-Parquet results all have the same 4 columns: `role_rank, role_category, credit_count, avg_billing_order`. The partition results have the same columns plus `partition_id` at the front.

A row is dropped if it does not have 6 fields, if the IDs do not match the tt/nm pattern, if ordering is below 1, or if category is empty or `\N`. Only rows with `billing_order <= 10` are counted. We group by `role_category`, count the rows, and average the ordering to 2 decimals. Ranking is by count from high to low, and ties go to the role name in A to Z order.

There are a few data-quality risks. IMDb writes missing values as `\N`, which can be read as normal text. Category names are not consistent, since some use underscores like `archive_footage`. The file could have CRLF line endings. The titles in `title.basics` have quote characters that can break a CSV reader.

Each stage has a check. After cleaning, the header has to be the 4 expected names and every row needs 4 fields with valid IDs. The Parquet file is built from the exact cleaned TSV with fixed column types. The three results have to match with `diff`. At the end every output needs the right header, at least one data row, and numbers in the right format.

The scheduled jobs use the release era of the title from `startYear` as the partition: before 1980, 1980 to 1999, 2000 to 2009, 2010 to 2019, and 2020 or later. The output a later project stage will reuse is `out/final_analysis.tsv`, which has the roles with at least 10 counted credits.

Stage map:

```
raw input (title.principals.tsv)
  -> validated and normalized TSV        (prepare_data.sh, awk + sed)
  -> equivalent Parquet representation   (build_parquet.sql)
  -> shell-text, DuckDB-text, and DuckDB-Parquet results
  -> equivalence check with diff, and timing (run_benchmark.sh)
  -> one scheduled era partition per day (cron + run_scheduled.sh)
  -> final project analysis artifact (out/final_analysis.tsv)
```

2. Making the text and Parquet data

`prepare_data.sh` uses `awk -F '\t'` to check the header, keep the 4 fields we need, and throw out rows that break the rules. It writes how many rows were read, kept, and dropped to the log. Then `sed` removes a trailing `\r` if there is one, renames the header to `title_id, billing_order, person_id, role_category`, and changes `archive_footage`, `archive_sound`, and `production_designer` to use hyphens. A second awk pass checks the field count and ID patterns again and exits with a nonzero status if anything fails.

`build_parquet.sql` reads that same TSV with `read_csv` and set column types, then writes `work/analysis_ready.parquet` with `COPY ... (FORMAT parquet)`. We set the types ourselves so DuckDB does not guess differently for the text and Parquet versions.

3. The three implementations

The shell version (`run_shell_analysis.sh`) uses awk to keep rows with ordering 10 or less and adds up counts and orderings per role in arrays. It prints the average with `%.2f`. Then `sort -t $'\t' -k2,2nr -k1,1` sorts by count and then by name, and one more awk adds the rank number.

The two DuckDB versions use the same query. `analysis_text.sql` reads the TSV and `analysis_parquet.sql` reads the Parquet file. The first CTE filters `billing_order <= 10`, the second one groups by role, and `ROW_NUMBER() OVER (ORDER BY credit_count DESC, role_category ASC)` gives the rank. The result is saved with `COPY ... (HEADER, DELIMITER '\t')`. We picked a CTE and a window function because the question is a ranking, and the rank has to be computed after grouping. Using the same tie rule as `sort` means the shell and SQL ranks line up.

4. Checking that the three results match

`run_pipeline.sh` runs `diff -u` on shell vs DuckDB-text and on DuckDB-text vs DuckDB-Parquet, saves both to `out/result_comparison.txt`, and exits with 1 if either one is different. We ran it on the 1,000-row sample, on the first 100,000 rows of the real file, and on the whole file. Every time both diffs were empty and it printed `[OK] All three results agree.` The benchmark also checks this on its 1,000,000-row input before timing anything.

On the full file, 78,291,836 credits were in the top 10 billing spots. Actor was first with 23,445,643 credits (average position 4.61), then actress with 17,815,652 (4.52), self with 15,345,823 (3.66), writer with 6,288,852 (5.90), director with 5,665,913 (4.83), and producer with 3,936,750 (6.10). After that came editor (1,998,284), cinematographer (1,655,275), composer (1,179,896), archive-footage (633,531), production-designer (182,988), casting_director (132,458), and archive-sound (10,771). One thing we noticed is that `casting_director` still has an underscore, because the sed step only fixes three names. That should be fixed next time.

5. Benchmark

`run_benchmark.sh` takes the header and the first 1,000,000 rows of `title.principals.tsv`, cleans them with the same `prepare_data.sh`, makes a Parquet file from that exact TSV, checks that the three results match, and then times each path 3 times with the bash `time` keyword. It runs inside `work/bench/` so it

does not overwrite the sample outputs we commit.

The shell version took 1.184, 1.187, and 1.169 seconds, about 1.18 on average. DuckDB on the text file took 0.282, 0.293, and 0.281 seconds, about 0.29. DuckDB on Parquet took 0.072, 0.076, and 0.094 seconds, about 0.08. All nine runs are marked `result_verified=yes` in `out/benchmark.tsv`.

The TSV was 30,269,468 bytes and the Parquet file was 6,603,243 bytes, so Parquet was a little over a fifth of the size. Making the Parquet file took 0.452 seconds, and that is not counted in the times above. For this input and this query, DuckDB on Parquet was the fastest, DuckDB on text was next, and the shell version was about 4 times slower than DuckDB on text.

When DuckDB read Parquet instead of text, the same query went from about 0.29 to 0.08 seconds. Our guess is that this is because Parquet already has typed columns, so there is no text to parse, and the file is smaller. Since each query saves about 0.2 seconds, the conversion pays for itself after 2 or 3 queries. The three runs of each path were close. The biggest gap was the third Parquet run at 0.094 seconds compared to about 0.074 for the other two.

A few things could affect these numbers. The server is shared, so other people's jobs can slow things down. Later runs might be faster because the file is already cached. The DuckDB runs are so short that starting the program is a big part of the time. This is one input and one query, so we are not saying DuckDB is always faster.

The SQL was easier to read and change because the filter, grouping, and tie rule are all in one query. In the shell version the same logic is split across awk, sort, and another awk, and the tie rule is hidden in the sort options. For grouped and ranked summaries like this, we plan to use DuckDB with Parquet in the next stage. We will keep the awk and sed cleaning step since it was simple and worked well.

6. Background run

We ran the full pipeline in the background with this command:

```
nohup bash pa3/scripts/run_pipeline.sh
/mnt/scratch/CS131_jelenag/projects/team06_sec02_fall2026/imdb_data/title.principals.tsv
> pa3/logs/full_run.log 2>&1 &
```

Its PID was 2696320. We watched it with `jobs -l`, `ps -o pid,etime,cmd -p $PID`, `tail` on the log, and by checking the size of `work/analysis_ready.tsv`, which reached 3.0 GB during stage 1. The log ended with `[OK] Pipeline completed and all outputs verified.` and bash showed `[1]+ Done`, which means it exited with status 0. The full proof is in `out/background_run_proof.txt`. Afterward we saved the full Parquet file for Task 9 and ran the sample again so the committed outputs are from the sample.

7. Partitions and scheduled runs

`build_partitions.sh` uses awk on `title.basics.tsv` to give every title an era based on field 6 (`startYear`) and skips titles where it is `\N`. DuckDB joins that list to the full analysis-ready Parquet file and writes one Parquet file for each era. The script also writes `state/partition_manifest.tsv` with each partition ID and its full path. We chose era because it is a real field in the data and it lets us see if the role mix changes over time, which matters when recommending older vs newer movies. The

partitions are not equal in size: before 1980 has 9,051,334 rows, 1980 to 1999 has 11,686,058, 2000 to 2009 has 15,005,667, 2010 to 2019 has 31,362,224, and 2020 or later has 25,050,029, for 92,155,312 in total. Even the biggest one finishes well before the next daily run.

`run_scheduled.sh` sets `PATH` and the full DuckDB path, moves into the `pa3` folder, and picks the first partition in the manifest that does not have a marker in `state/completed/`. It runs `run_partition.sh` on it and adds a line to `logs/cron.log` with the time, partition, path, start and end times, and exit status. It only creates the `.done` marker if the exit status is 0, so a finished partition is never run again. When all partitions are done it logs a no-work message and exits with 0. Our cron entry is `35 4 * * *`, which is 4:35 AM every day in server time (EDT).

The first scheduled run was at `2026-10-27 04:35:08`. It processed `era_1980_1999` with `exit_status=0` and created `era_1980_1999.done`. [Run 2 on 2026-10-28 at 04:35 EDT: add the partition and exit status if it ran before submission.] We only finished the pipeline one day before the due date, so we could not get three daily runs in time. `out/scheduling_proof.txt` only shows the runs that really happened. We did not speed up the schedule or edit the log. The cron entry [was removed on ____, checked with crontab -l].

8. A failure and how we figured it out

On our first try at the full run, we started the sample pipeline while the background run was still on stage 1. The sample run failed with `[ERROR] Output verification failed on 'work/analysis_ready.tsv'`, and the file we copied as the full result had the sample's numbers (director 418). When we ran `ps -o pid,etime,cmd`, the full run was only 21 seconds old and still on stage 1. Both runs had been writing to the same `work/analysis_ready.tsv`, so they messed up each other's file.

We stopped the background run with `kill -TERM`. But `ps -o pid,ppid,etime,cmd -u $USER` showed `prepare_data.sh` and its `awk` and `sed` were still running with parent PID 1, so killing the main script did not stop the processes it started. We killed those three PIDs, deleted the broken files, started the run again, and did not touch `pa3` until `ps` showed it was done. Next time we would add a check at the start of `run_pipeline.sh` so it will not start if another run is already using `work/`.

9. What out/final_analysis.tsv is for

`final_analysis.tsv` has the roles with at least 10 counted credits, ranked by count, with their average billing position. In the final project we will use it to decide which roles become recommendation features. Common, high-billed roles like actor, actress, and director will be features, and rare roles will be dropped or grouped together. The committed file is from the sample. The full-data numbers are in Section 4.

10. Next technical step

Join the cleaned credits with `title.ratings.tsv` on `title_id` to compare the roles in highly rated titles vs low-rated ones, and do it per era partition. We also want to fix the sed step so every category name is cleaned the same way.

11. Team contributions

Prajwal did Tasks 2, 3, 4, 5, and 7. He wrote `prepare_data.sh`, the shell and DuckDB analyses, the Parquet build, the diff check, and `run_pipeline.sh` with the sample outputs. Pouya moved the sample file to the required path `pa3/data/sample/sample.tsv`. [add anything else Pouya did] Aneesh did Tasks 6, 8, and 9. He wrote the benchmark and ran the nine timed runs, did the background full run and proof, built the era partitions, set up the scheduled job and cron, and put together this report. Krishna [fill in]. Hadi [fill in].

12. AI-use disclosure

Aneesh used Claude (Anthropic) for help with Tasks 6, 8, 9, and 10. Claude wrote first drafts of `run_benchmark.sh`, `build_partitions.sh`, `run_partition.sh`, `run_scheduled.sh`, `build_partitions.sql`, `analysis_partition.sql`, and the cron file, and was told to stick to tools from the course slides. It also gave the commands to run on the server, helped read the output, and drafted this report from our results. Claude also walked us through the cron part of Task 9: what the five schedule fields mean, how to install the entry with `crontab` and check it with `crontab -l`, that cron uses the server's time zone (EDT) and not ours, and how to remove only our entry when we are done. Aneesh ran every command on the IBM server and checked the outputs. All numbers in this report come from our own runs. [Other team members: add any AI use for your parts.]